

PATH GUIDING ROBOT

By

Maulik Pithiya (23BSIT119)

Sujal Pandya (23BSIT055)

Abhishek Parmar (23BSIT060)

Under Guidance

of

Internal Guide

Dr. Shailesh Khant

Submitted to



Smt. Chandaben Mohanbhai Patel Institute of Computer
Applications
CHARUSAT
Changa

April 2026



Accredited with Grade A+ by NAAC,
CHAROTAR UNIVERSITY OF SCIENCE AND
TECHNOLOGY
Changa

Acknowledgement

Knowledge in itself is a continuous process. At this moment of our substantial enhancement, we rarely find words to express our gratitude towards those who were constantly involved with us.

The completion of any inter disciplinary project depends upon coordination, cooperation and combined efforts of several resources of knowledge, creativity, skill, energy and time. The work being accomplished now, we feel our sincerest urge to recall and knowledge through these lines, trying our best to give full credit wherever it deserves.

We would like to thank our project guide **Dr. Shailesh Khant**, Principal **Dr. Dharmendra Patel** and Dean **Dr. Sanskruti Patel** who advised and gave us moral support through the duration of our project. Without their constant encouragement we could not have been able to achieve what we have.

It's our good fortune that we had support and well wishes of many. We are thankful to all and those names which have been forgotten to acknowledge here but contributions have not gone unnoticed.

With Sincere Regards,

Maulik Pithiya(23BSIT119)

Sujal Pandy (23BSIT055)

Abhishek Parmar(23BSIT060)

Table of Contents

Sr. No		Subject	Page Number
1		Project Profile & Company Profile	1
2		Introduction to tools	3
3		System Study	6
	3.1	Existing System	7
	3.2	Proposed System	8
	3.3	Scope of the Proposed System	14
	3.4	Aim and Objective of the Proposed System	15
	3.5	Feasibility Study	17
	3.5.1	Operational Feasibility	17
	3.5.2	Technical Feasibility	17
	3.5.3	Economical Feasibility	18
4		System Analysis	20
	4.1	Requirements Specification (along with System Modules)	21
	4.2	Use Case Diagram	23
	4.3	Activity Diagram	24
	4.4	IOT Architecture Diagram	25
	4.5	Block Diagram	26
5		System Design	27
	5.1	Data Dictionary	28
	5.2	Screen Layouts	30
	5.3	Reports	37
6		System Testing	38
	6.1	Testing Strategies	39
	6.2	Test Cases	40
7		Future Enhancement	43
8		Bibliography/References	46
9		Reporting Report (to be collected from respective internal guide)	48

PROJECT PROFILE

❖ Project Profile

Project Name: Path Guiding Robot

Type of Application: Internet of things

Project Description: The Path Guiding Robot is an autonomous line-following robot designed to navigate a fixed campus environment and guide users from one location to another. The system integrates embedded hardware, real-time wireless communication, and a web-based user interface to deliver a seamless navigation experience.

Team Size: 3

Front End: HTML5, Tailwind CSS, JavaScript

Back End: FireBase Realtime Database, FireStore

Tools used: Tinkercad, Wokwi, Cloude, VS Code, Arduino IDE, GitHub, Vercel

INTRODUCTION TOOLS

❖ **Introduction to Tools**

➤ **Front End Tool:**

The project includes Web base application. This WebApp allows users to send commands, monitor the robot's live status, and track its real-time location.

The front end of the application is developed using HTML for structuring the interface, Tailwind CSS for responsive and modern styling, and JavaScript for implementing interactive features and application logic.

HTML5

It is a 5th and latest version of Hypertext markup language (HTML), helps to create a structure of web-page.

Tail-Wind CSS

It is utility-first approach to styling, meaning you build designs by combining small, single-purpose classes directly in your HTML rather than relying on prebuilt components or writing custom CSS.

Java Script:

It is a versatile, high-level programming language that allows websites to be interactive and dynamic by enabling features such as form validation, animations, real-time updates, and communication with servers without reloading the page. It runs directly in the browser.

➤ **Back End Tool:**

Firebase Realtime Database

It is a cloud-hosted NoSQL database provided by Google as part of the Firebase platform. It stores and synchronizes data in JSON format across all connected

clients in real time. Unlike traditional databases that require explicit fetch requests, Firebase Realtime Database uses a continuous connection to push data updates instantly.

In the proposed system, Firebase Realtime Database serves as the primary communication bridge between the RoboNav web interface and the ESP32 microcontroller. Firebase Realtime Database was chosen for this role due to its low latency, free tier availability, and native support in the ESP32 Arduino ecosystem through the FirebaseESP32 library.

Firebase Firestore

Cloud Firestore is a flexible, scalable NoSQL document database also provided by Google as part of the Firebase platform. Unlike Firebase Realtime Database which stores data as a single large JSON tree, Firestore organizes data into collections and documents, making it more suitable for structured and query able data.

In the proposed system, Firestore is used exclusively for storing visitor navigation logs. Each time a user initiates navigation, a new document is created in the visitor's collection containing the visitor's name, source node, destination node, building names, path taken, and timestamp. The use of Firestore for this purpose allows efficient querying and filtering of visitor data for analytics purposes.

SYSTEM STUDY

3.1 Existing System

In the past, path-following robots used older and less advanced technology. They usually worked with small microcontroller boards like the Arduino (ATmega328), which don't have much processing power and can't connect wirelessly. These robots often relied on one or two infrared (IR) sensors that only checked if the robot was on a dark or light surface. Because of this, they could follow very basic paths but struggled with more complicated routes or detecting intersections (nodes).

The existing systems generally exhibit the following limitations:

- **Fixed Destination Only:** Most existing path guiding robots are programmed for a single fixed destination and cannot dynamically change the target based on user input at runtime.
- **No Web Interface:** Existing systems typically use physical buttons mounted on the robot itself for input, making it inconvenient for users who are unfamiliar with the robot.
- **Complex Hardware:** Many existing path guiding robots use expensive sensors like LiDAR, ultrasonic arrays, or cameras for navigation, making them costly and difficult to maintain.
- **No Real-Time Status Monitoring:** Existing systems do not provide any remote monitoring of robot status, current position, or navigation progress to the user.

These shortcomings make the existing systems unsuitable for modern industrial or educational applications that demand reliability, feedback, and flexibility.

3.2 Proposed System

The proposed system is divided into two major components software and hardware. On the software side, a web-based interface called RoboNav allows the user to select a source and destination node from a fixed campus map.

Upon clicking Start Navigation, the user is prompted to enter their name which is logged into Firestore for analytics. Then Breadth-First Search algorithm computes the shortest path between the selected nodes and generates a step-by-step direction sequence.

On the hardware side, the ESP32 microcontroller connects to Wi-Fi and continuously listens to the Firebase Realtime Database for new navigation commands. Once a command is received, the robot begins execution.

A 5-array IR sensor module detects the black line on the ground and provides digital readings to the ESP32. A PID control algorithm processes these sensor readings to calculate real-time motor corrections, ensuring smooth and accurate line following. At each node, detected when all five sensors simultaneously read the line, the robot executes the next direction from the received sequence. Motor speed and direction are controlled through the L298N motor driver which receives PWM signals from the ESP32. An OLED display provides real-time status feedback including Wi-Fi connection status, IP address, current direction and other details. All connections between components are routed through a breadboard making the platform modular and easy to modify during development.

To provide real-time visibility of the robot's operational state, the ESP32 microcontroller continuously writes its current status back to the Firebase Realtime Database throughout the navigation process. This bidirectional communication ensures that the RoboNav web interface always reflects the latest state of the robot without requiring any manual refresh.

When the ESP32 powers on and successfully connects to Wi-Fi, it immediately updates the connection status to Online in the database, which is reflected on the website as a live indicator. Each status change is written to Firebase in real time and the website listens for these changes and updating the display instantly.

This live status feedback allows the user to monitor the robot's progress from the web interface at any point during navigation, making the system fully transparent and interactive from both ends.

Table 3.1: The key hardware components and their roles

Component	Role in Proposed System
ESP32 Microcontroller	Central processing unit; handles sensor reading, motor control logic, display updates, and future wireless communication.
L298N Motor Driver	Takes signals from the ESP32 and drives the two DC motors with proper power and speed control.
5 Array IR Sensor	Sensor array enabling accurate detection of the line path, including sharp curves and intersections, via sensor signal.
OLED I2C Display	Displaying sensor status, motor speed, and other details to the user during runtime.
Li ion Battery	Portable power supply providing sufficient current for hardware components.

3.2.1 Connections

This section describes the hardware connections and component layout of the proposed path guiding robot system.

Table 3.2: IR Sensor Array (5-Array Digital)

Pin No.	Pin Name	Used For	Status
GPIO 34	IR1	Far Left signal input	Input
GPIO 35	IR2	Left signal input	Input
GPIO 32	IR3	Center signal input	Input
GPIO 19	IR4	Right signal input	Input
GPIO 18	IR5	Far Right signal input	Input
VIN	5V	VCC Power Supply (5V) Motor Driver	Power
GND	GND	Ground	Power

Note: 0 = Black Line detected, 1 = White Surface.

Table 3.3: L298N Motor Driver

Pin No.	Pin Name	Used For	Status
GPIO 14	ENA	PWM speed control for Left Motor	Output
GPIO 26	IN1	Direction control pin 1 for Left Motor	Output
GPIO 27	IN2	Direction control pin 2 for Left Motor	Output
GPIO 12	ENB	PWM speed control for Right Motor	Output
GPIO 25	IN3	Direction control pin 1 for Right Motor	Output
GPIO 33	IN4	Direction control pin 2 for Right Motor	Output
GND	GND	L298N Motor Driver - Common Ground	Ground

Table 3.4: I2C OLED Display (0.96 inch SSD1306)

Pin No.	Pin Name	Used For	Status
GPIO 21	SDA	I2C Data Line	Output
GPIO 22	SCL	I2C Clock Line	Output
3.3V	3.3V	VCC Power Supply (3.3V)	Power
GND	GND	Ground	Ground

Table 3.5: Power Supply Connections

Pin Name	Used For	Status
VIN	Li-Ion Battery via Switch - ESP32 Power Input	Power
GND	Common Ground - Battery, ESP32, L298N and IR Sensor	Ground
3.3V	OLED Display Power - from ESP32 onboard 3.3V	Power
5V	IR Sensor Module Power - from ESP32 VIN pin	Power

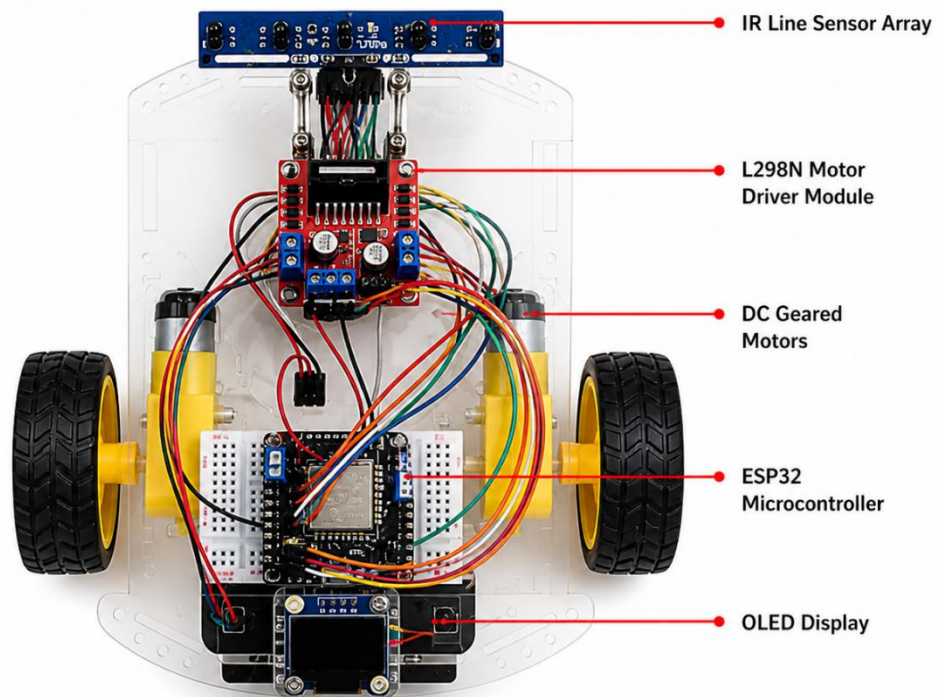


Figure 3.1: Top View of Path Guiding Robot Hardware Setup

Description:

This figure shows the hardware setup of the proposed path guiding robot system. It includes key components such as the ESP32 microcontroller, IR line sensor array for path detection, L298N motor driver module for motor control, DC geared motors for movement, and an OLED display for status visualization. These components work together to enable real-time navigation and path following.

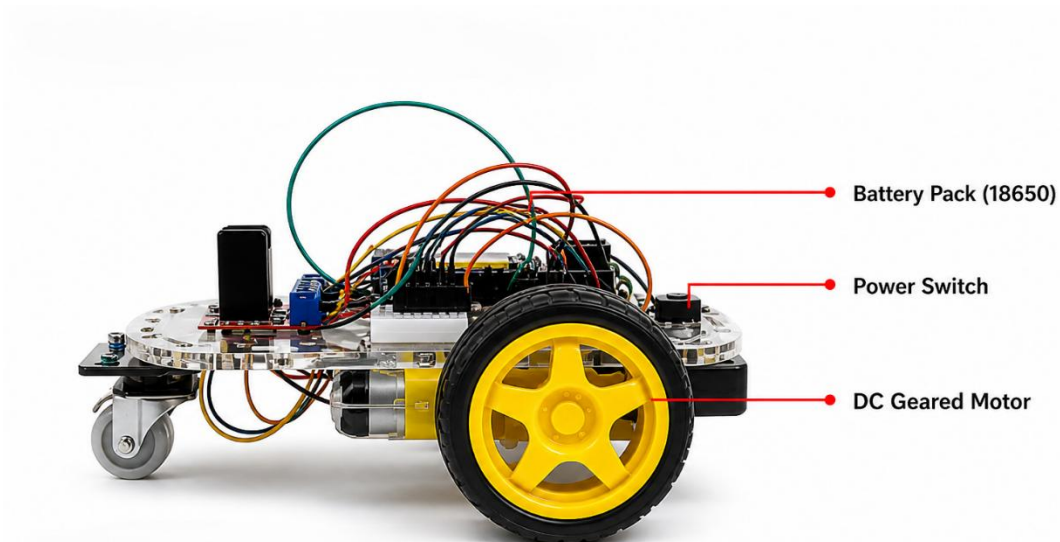


Figure 3.2: Side View of Path Guiding Robot Hardware Setup

Description:

This figure shows the bottom/side view of the robot highlighting the power and drive components. It includes the 18650 battery pack for power supply, a power switch for system control, and DC geared motors responsible for movement.

3.3 Scope of the Proposed System

The system is capable of computing the shortest path between any two selected nodes using the Breadth-First Search algorithm and guiding the robot along a physical line track. The robot can navigate straight paths, gradual curves, and 90-degree turns at junctions based on the direction sequence received from the web interface.

The scope of the system includes:

- A web-based user interface accessible from any device for selecting source and destination nodes
- Shortest path computation and direction generation on the website
- Wireless communication between the web interface and the robot via Firebase Realtime Database
- Autonomous line following using PID control with a 5-array IR sensor
- Node detection and direction execution at junctions
- Real-time robot status updates visible on the web interface
- Visitor logging and campus analytics through a dedicated admin dashboard

The system does not cover navigation outside the predefined campus map and operation without a Wi-Fi connection.

The physical track used in the current implementation is a simplified representation of the campus layout of CHARUSAT, consisting of a black line on a white surface. The system is intended as a proof-of-concept deployment suitable for indoor campus navigation guidance.

3.4 Aim and Objective of the Proposed System

3.4.1 Aim

The aim of the proposed system is to develop a low-cost, autonomous path guiding robot for fixed path but dynamic destinations, assists visitors in navigating a campus environment by providing a simple web-based interface for destination selection and guiding the robot to the desired location through intelligent line following and real-time wireless communication.

3.4.2 Objectives

- To design and develop a web-based user interface that allows users to select a source and destination location from a predefined campus map and initiate robot navigation.
- To implement the Breadth-First Search algorithm on the web interface to compute the shortest path between any two selected campus nodes and generate a step-by-step direction sequence.
- To establish reliable real-time communication between the web interface and the ESP32 microcontroller using Firebase Realtime Database, enabling instant transmission of navigation commands and status updates.
- To implement a PID control algorithm on the ESP32 for smooth and accurate line following using a 5-array IR sensor module, ensuring stable robot movement on straight paths and curves.
- To develop node detection logic that enables the robot to identify decision points on the track and execute the correct direction Forward, Left, Right, or Stop based on the received direction sequence.

- To integrate an OLED display on the robot for local status indication, showing Wi-Fi connectivity, IP address, current navigation direction, and error conditions during operation.
- To log each navigation event along with visitor details to Firestore Database and provide an admin analytics dashboard displaying meaningful insights such as most visited buildings, popular routes, unique visitor count, and building popularity statistics.
- To demonstrate a complete end-to-end proof-of-concept system that integrates web development, cloud database, embedded programming, and robotics into a single cohesive solution applicable to real-world campus navigation.

3.5 Feasibility Study

A feasibility study was conducted to evaluate whether the proposed Path Guiding Robot system is practical and achievable within the given constraints of time, resources, and technical knowledge. The study was carried out across three dimensions Operational, Technical, and Economical.

3.5.1 Operational Feasibility

The proposed system is operationally feasible as it addresses a real and practical need guiding visitors across a campus environment without requiring human assistance. The web interface is simple and intuitive, requiring only two inputs from the user: a source location and a destination. visitor input name which adds minimal friction to the user experience.

The system operates autonomously once navigation is initiated, requiring no further interaction from the user. The OLED display on the robot provides local feedback while the website provides remote status monitoring, making the system transparent and easy to use for any visitor regardless of technical background.

The admin analytics dashboard provides meaningful insights to campus authorities about visitor patterns and building popularity, adding operational value beyond just navigation. Overall, the system is designed to be user friendly, low maintenance, and practically deployable in a campus environment or warehouse.

3.5.2 Technical Feasibility

The proposed system is technically feasible as all components and technologies used are well established, widely available, and within the technical competency of the development team.

On the hardware side, the ESP32 microcontroller provides built-in Wi-Fi capability eliminating the need for additional networking hardware module.

The L298N motor driver, 5-array IR sensor, OLED display, and DC gear motors are all low-cost and widely used components.

The PID control algorithm and node detection logic are implementable within the processing capacity of the ESP32.

On the software side, the web interface is built using standard HTML, Tailwind CSS, and JavaScript requiring no advanced frameworks. Firebase Realtime Database and Firestore are cloud-based services that provide free tier access sufficient for the scope of this project.

The integration of all components through Firebase as a communication bridge is a proven IoT architecture pattern. All technologies used in this project were successfully tested and validated during the development phase, confirming technical feasibility.

3.5.3 Economical Feasibility

The proposed system is economically feasible as it is developed using low-cost, readily available components and free cloud services. The total hardware cost of the system is significantly lower than commercially available campus navigation solutions, making it an super affordable.

The software components including the web interface, Firebase Realtime Database, and Firestore are available under free tier plans which are sufficient for the scale of this project. The web application is hosted on Vercel which also provides a free hosting tier with automatic deployment.

The development was carried out using open-source tools including the Arduino IDE, freely available ESP32 libraries, and standard web technologies, incurring no software licensing costs. The overall investment is limited to the hardware components.

Table 3.6: The hardware components with cost

Component	Qty	Cost (₹)
ESP32 Dev Module	1	450
L298N Motor Driver	1	120
5 Channel IR Sensor Array	1	250
OLED I2C Display (0.96")	1	200
DC Gear Motors (with wheels)	2	200
Chassis / Robot Body	1	200
Battery (Li-ion)	1	150
Breadboard	1	100
Jumper Wires	-	50
TOTAL ESTIMATED COST		1720

SYSTEM ANALYSIS

4.1 Requirements Specification

This section describes the system requirements grouped according to major modules of the IoT-Based Line-Following Campus Navigation Robot. The system is divided into functional modules including User Interface, Cloud Communication, Navigation, and Hardware Control. Each module performs a specific role in the overall system operation.

Table 4.1: System Modules

Module	Description
Web Interface Module	The RoboNav web app (HTML, Tailwind CSS, JavaScript) where the user selects source and destination. Runs the BFS algorithm and displays the route on the campus map.
Firebase Module	Acts as the real-time communication bridge between the web app and the robot. Stores the navigation command, robot status, and current node.
ESP32 Wi-Fi Module	The ESP32 connects to Firebase over Wi-Fi, reads the latest command, and triggers the navigation logic.
Line Following Module	Reads the 5-array IR sensor data continuously and uses PID control to keep the robot centred on the line.
Path Navigation Module	Implements BFS on the robot side to determine turns at each intersection based on the pre-mapped campus graph.
Display & Feedback Module	Updates the OLED display with current status and controls the two LEDs based on robot state.

Motor Control Module	Sends PWM signals to the L298N motor driver to control speed and direction of both DC motors.
Admin Control Module	A dedicated administrative panel providing an Analysis Dashboard to monitor performance metrics (total navigations, unique visitors), building popularity via pie charts, and a detailed Recent Visitors log for tracking real time and historical navigation data.

4.2 Use Case Diagram

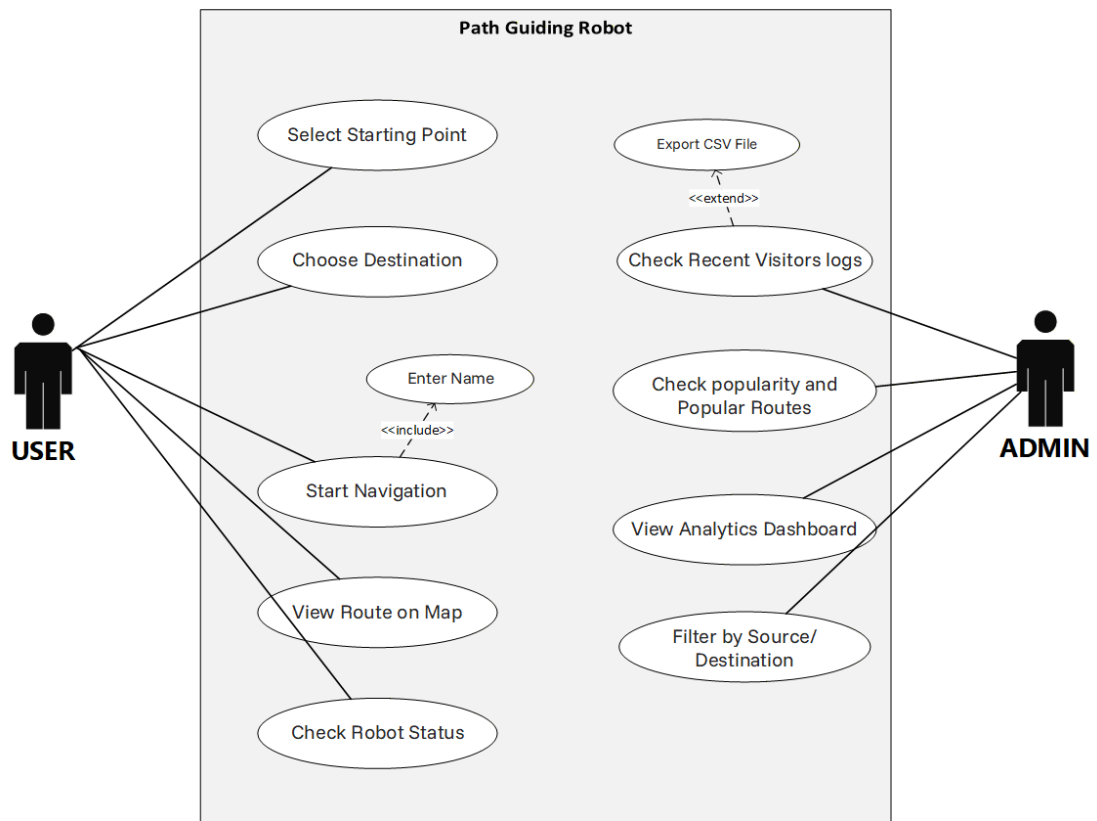


Figure 4.1: Use Case Diagram of Path Guiding Robot

4.3 Activity Diagram

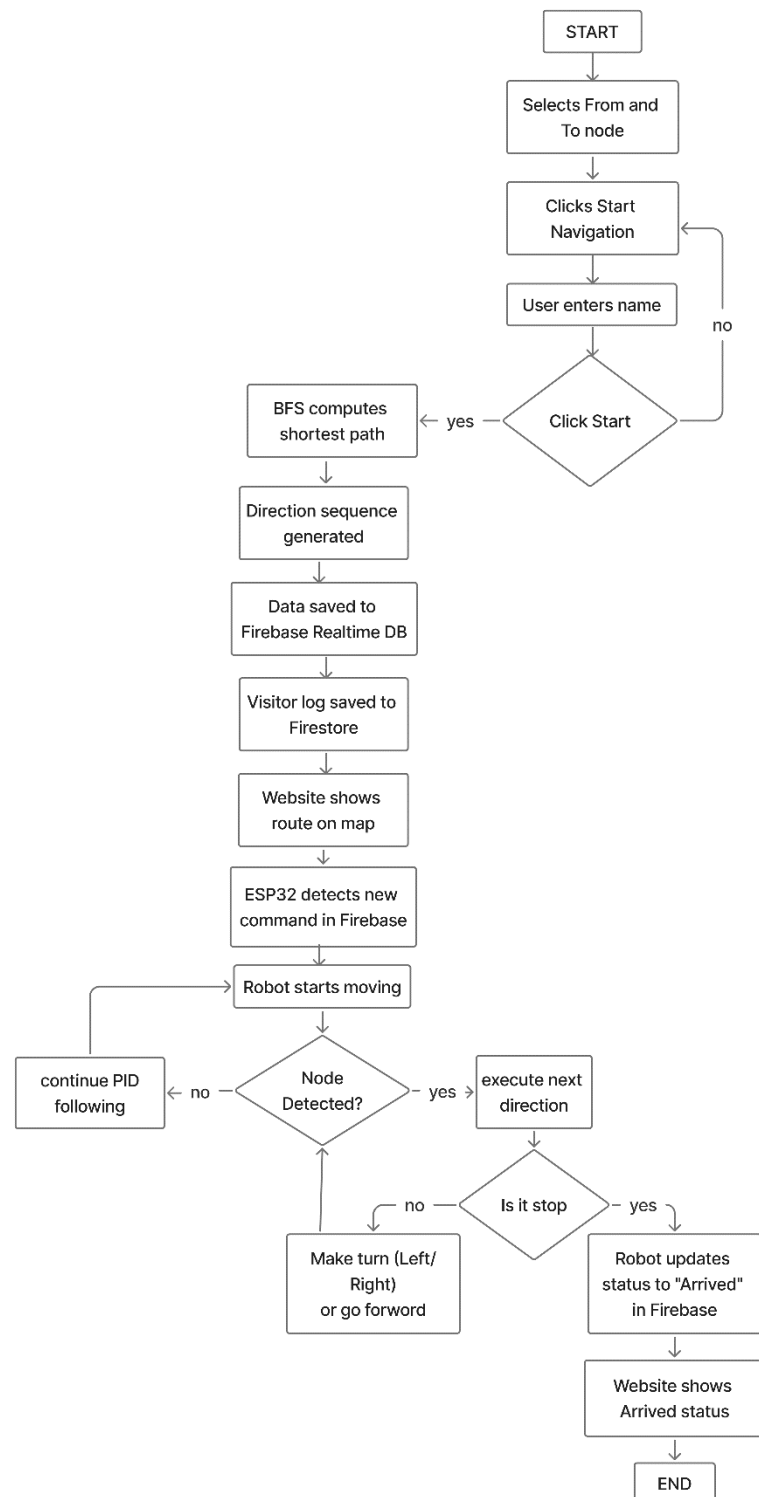


Figure 4.2: Activity Diagram for Path Guiding Robot

4.4 IOT Architecture Diagram

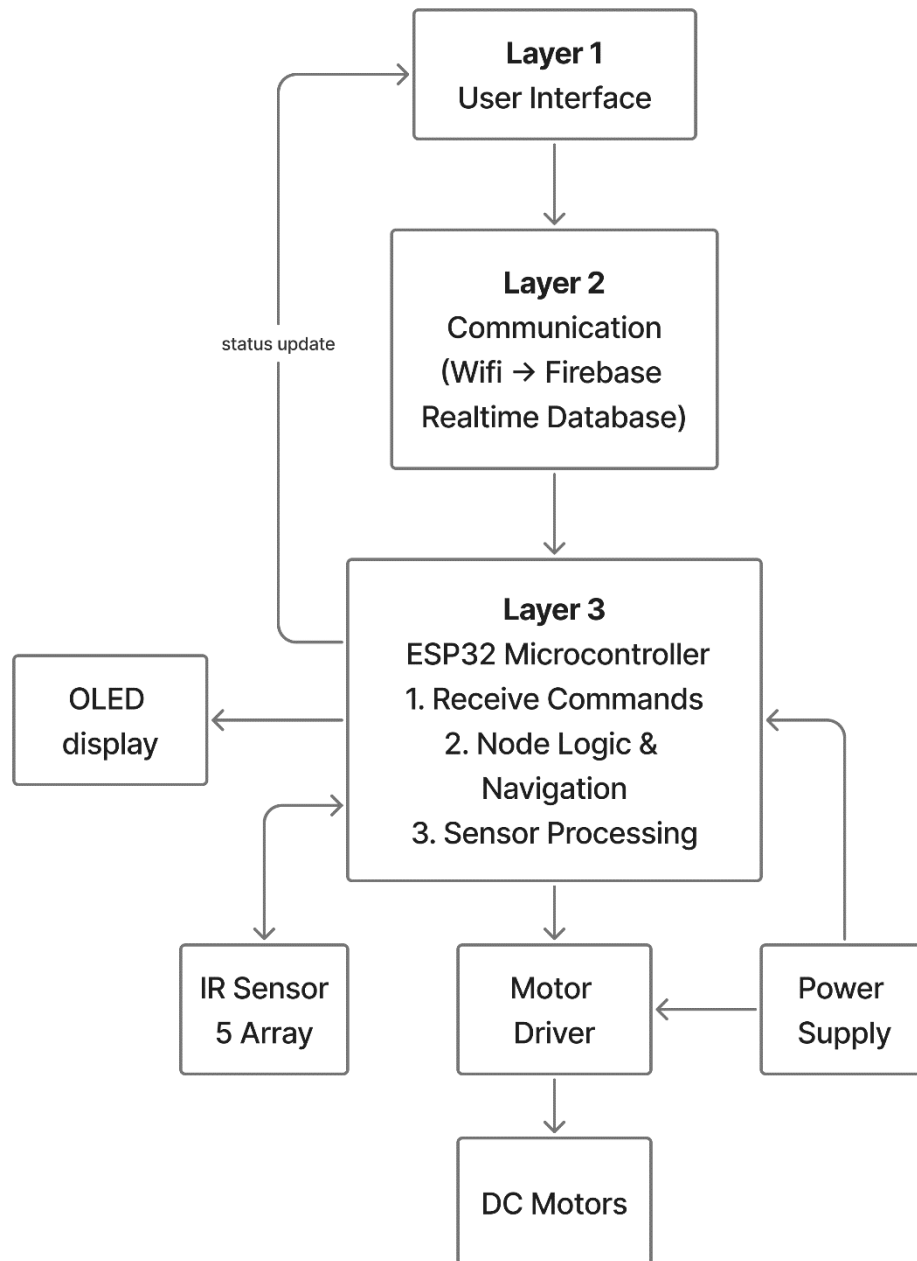


Figure 4.3: IoT Architecture of Path Guiding Robot System

4.5 Block Diagram

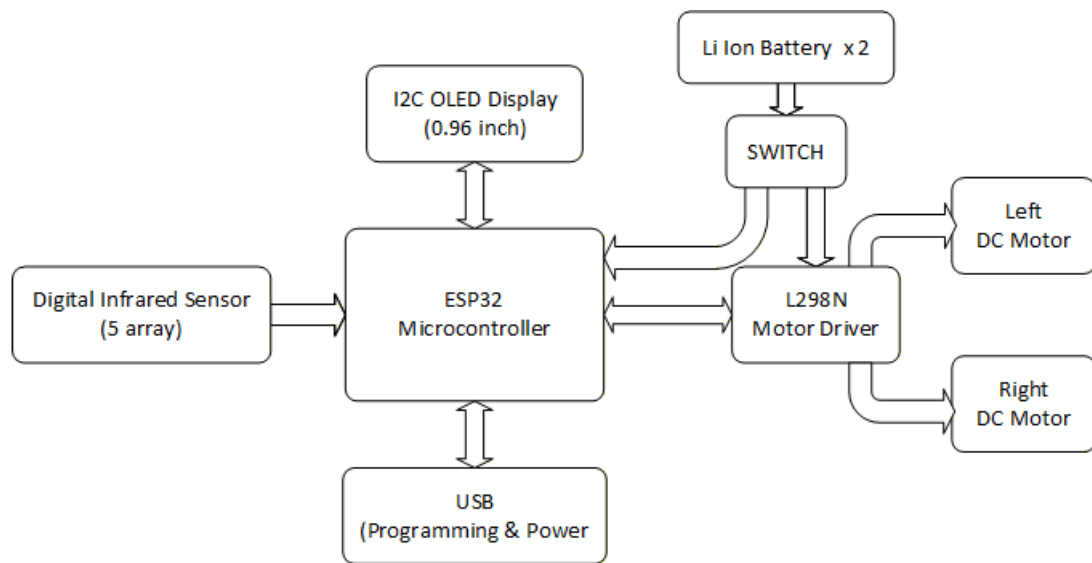


Figure 4.4: Block Diagram of Path Guiding Robot System

SYSTEM DESIGN

5.1 Data Dictionary

The data dictionary defines all the key data elements used in the system, their types, and their purpose.

Table: 5.1: Realtime Database Structure

Field Name	Data Type	Example	Description
esp32/lastHeartbeat	Number	18057	A rolling integer generated by the ESP32 to signal it is online.
directions	String	Forward, Left, Stop	Instructions generated by BFS for robot
from	String	N6	Starting node
path	String	N1, N3, N4	The nodes which will be included in path
status	String	pending	Current state: pending, moving, or reached
timestamp	Number	1777402613687	Timestamp when the command was issued.
to	String	N4	Destination node.

Table 5.2: Firestore Structure

Field Name	Data Type	Example Value	Description
from	String	N1	Node ID representing the point of origin.
fromName	String	MTIN	Name of the starting building.
name	String	Maulik	Name entered by user at the time of navigation.
path	Array	N1, N3, N4, N7	Ordered list of nodes retrieved from the BFS algorithm.
timestamp	Timestamp	April 2026 at 23:57:08 UTC+5:30	Precise date and time the visitor log was created.
to	String	N7	Node ID representing the final destination.
toName	String	Library	Name of the target building.

5.2 Screen Layouts

5.2.1 Home Page

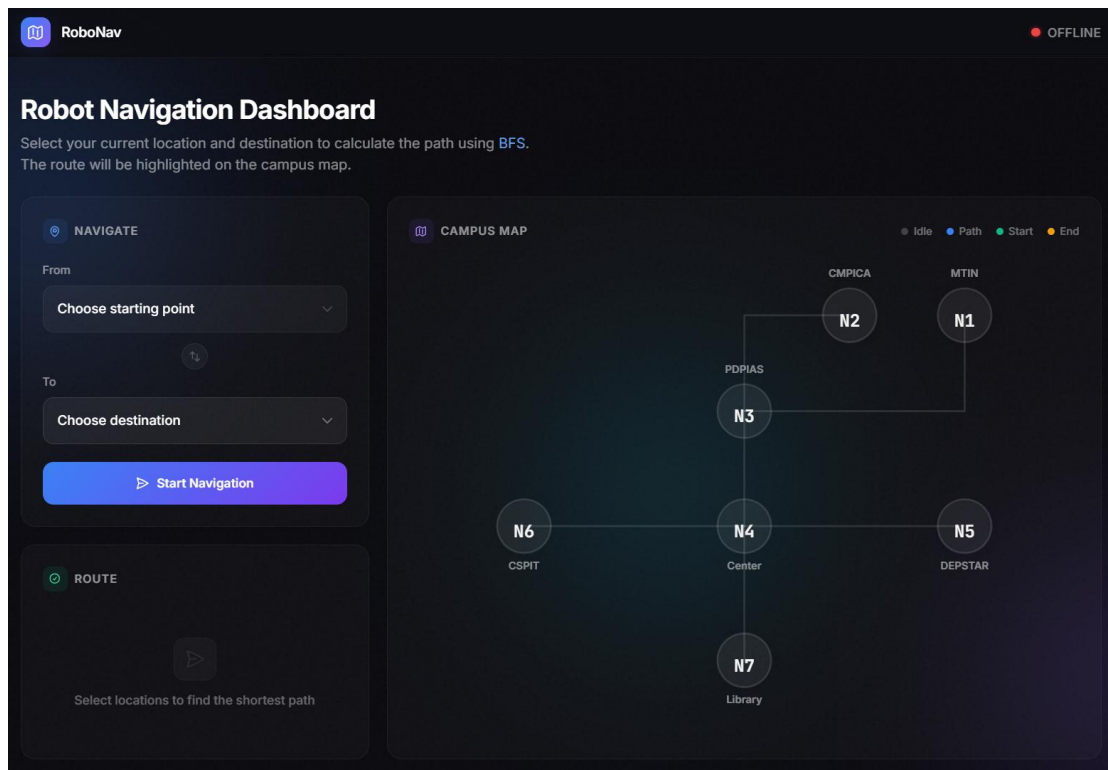


Figure 5.1: Home Screen of Path Guiding Robot System

Description:

The home screen consists of from/to dropdown menus, a Start Navigation button, a campus map graph, a route display section, and a robot status panel.

5.2.2 Name Input Popup

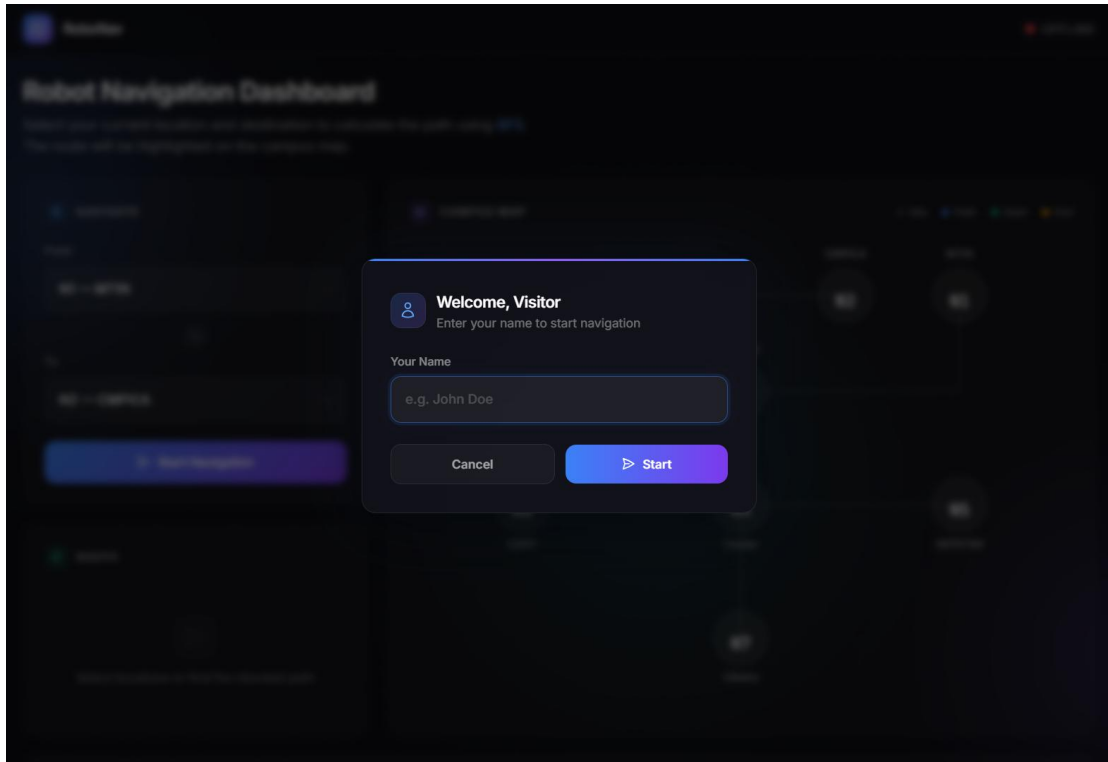


Figure 5.2: Name Input Popup Screen of Path Guiding Robot System

Description:

This screen displays a popup dialog prompting the user to enter their name before starting navigation. It includes a text input field for entering the user's name, along with "Start" and "Cancel" buttons to proceed or exit the navigation process.

5.2.3 Active Navigation Screen

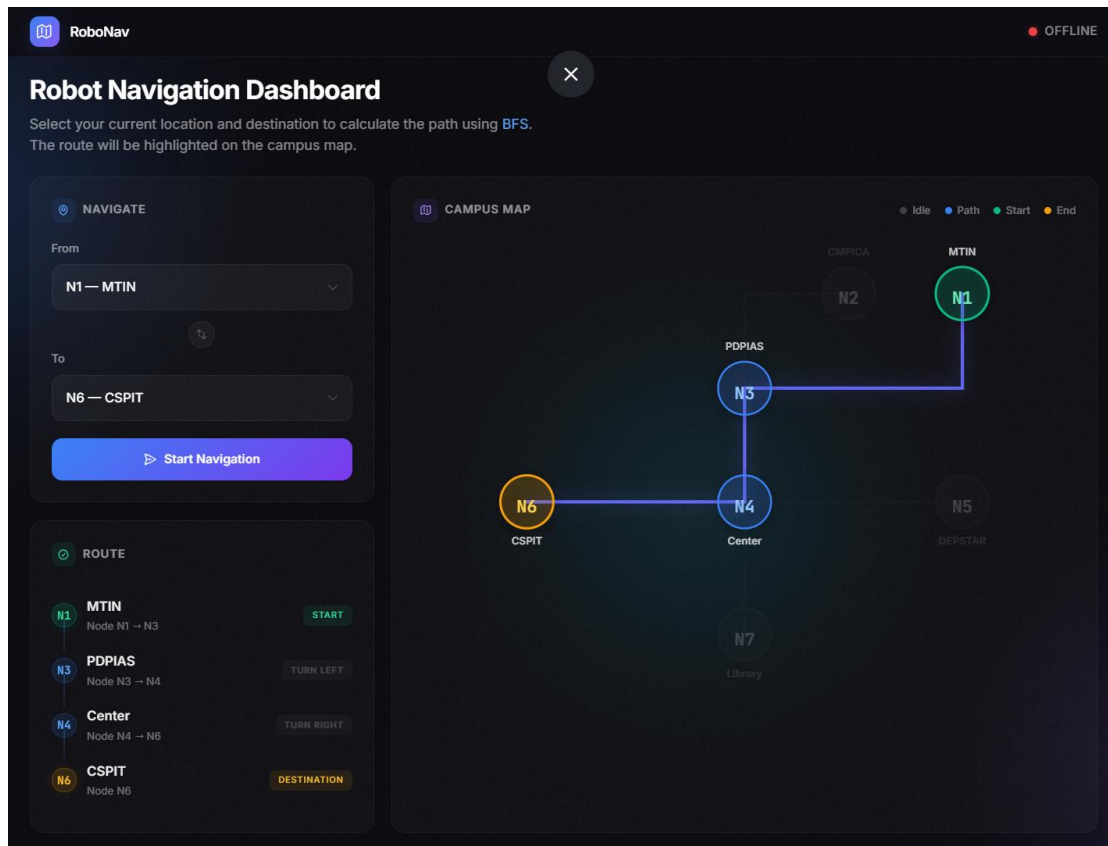


Figure 5.3: Navigation-in-progress screen

Description:

This screen displays the active navigation process of the robot. The computed path is highlighted on the campus map, showing the route from the starting point to the destination. It also includes step-by-step navigation instructions, along with real-time updates of the robot's current position and status.

5.2.4 Robot Status Panel

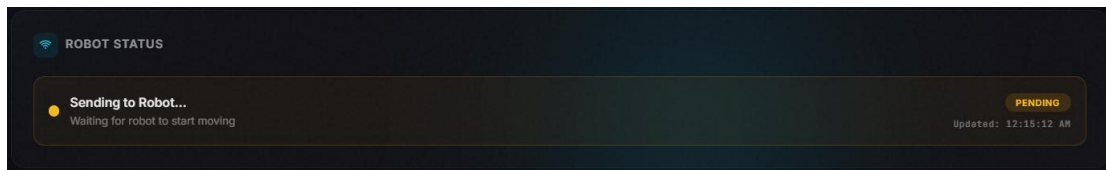


Figure 5.4: Robot Status Panel – Pending State

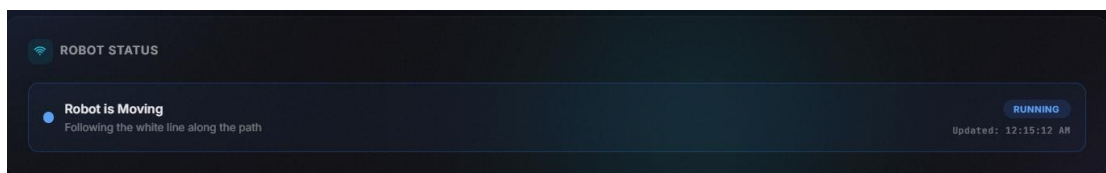


Figure 5.5: Robot Status Panel – Running State

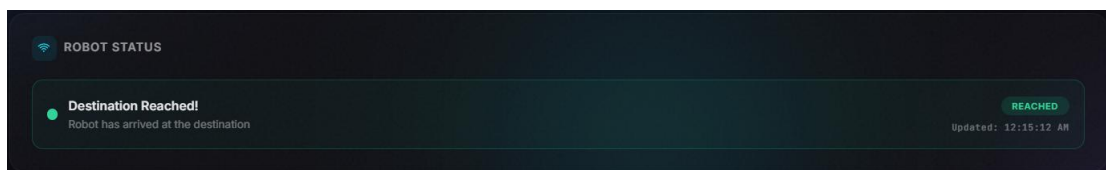


Figure 5.6: Robot Status Panel – Destination Reached State

Description:

This screen represents the robot status panel, which displays real-time updates of the robot during the navigation process. It shows different states such as pending (waiting to start), moving (actively navigating), and destination reached (navigation completed). These status updates help users monitor the progress of the robot throughout the navigation lifecycle.

5.2.5 Admin Analytics

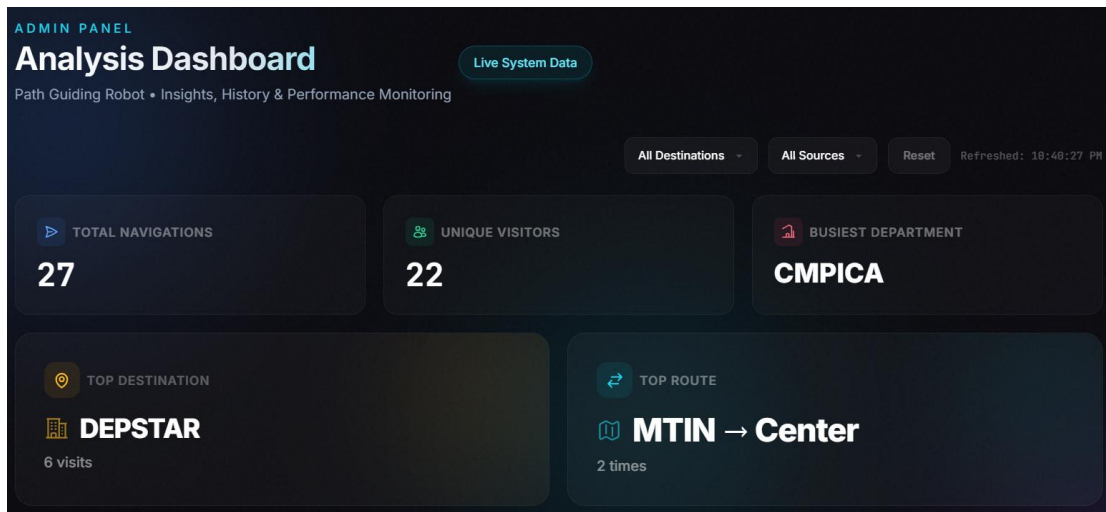


Figure 5.7: Admin Analysis Dashboard

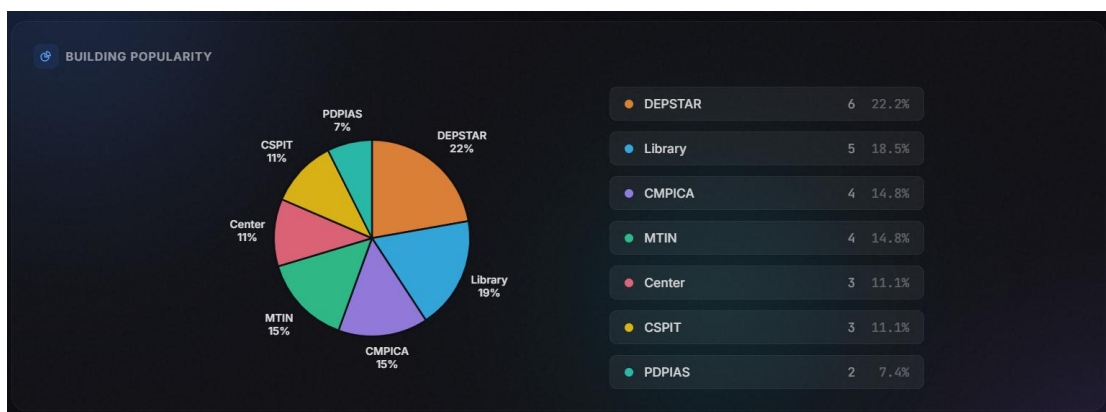


Figure 5.8: Building Popularity Pie Chart



Figure 5.9: Most Popular Routes Bar Chart

RECENT VISITORS 10 OF 30					Search by visitor name...	Export CSV
#	NAME	FROM	TO	TIME		
1	Jala	MTIN B1	Center B4	12:25 AM MAY 01, 2026		
2	Jenish	Center B4	MTIN B1	12:24 AM MAY 01, 2026		
3	Raja	MTIN B1	CSPIT B6	12:15 AM MAY 01, 2026		
4	Demo	MTIN B1	Center B4	10:07 PM APR 30, 2026		
5	Vishwa	MTIN B1	CSPIT B6	01:26 PM APR 30, 2026		
6	Sujal Pandya1234	Center B4	Library B7	01:26 PM APR 30, 2026		
7	Maulik	Center B4	CMPICA B2	10:59 AM APR 30, 2026		
8	Priyal	Center B4	CSPIT B6	10:34 AM APR 30, 2026		
9	Het	CSPIT B6	Center B4	12:26 AM APR 29, 2026		
10	Ravi	MTIN B1	Center B4	12:26 AM APR 29, 2026		

Figure 5.10: Recent Visitors Data Table

Description:

This screen represents the admin dashboard used for monitoring and analyzing system usage. It displays key statistics such as total navigations and unique visitors, along with visual insights including building popularity (pie chart) and most frequently used routes

(bar chart). It also includes a table of recent visitor logs, allowing administrators to track user activity and system performance.

5.3 Reports

The proposed Path Guiding Robot system generates three key reports during its operation. These reports serve as the primary outputs of the system, providing navigation guidance to the user, real-time operational feedback, and analytical insights to the administrator. Each report is described in detail below.

Table 5.3: System Generated Reports

Report Type	Description
Navigation Path Report	After clicking Navigate, the web app displays the complete path (e.g., N1 → N4 → N6) and highlights it on the campus map.
Robot Status Report	The Robot Status panel on the web app shows the current state, current node, and last updated time updated live from Firebase.
Visitor Analytics Report	Generated from Firestore visitor logs which includes Total navigations count, visitors count, Most popular route, Building popularity pie chart, Top 5 routes bar chart, Recent visitors table

SYSTEM TESTING

6.1 Testing Strategies

Testing was carried out at multiple levels to make sure every part of the system works correctly, both individually and together. We followed a simple, structured testing approach suited to an IoT and embedded project.

6.1.1 Unit Testing

Each component was tested in isolation before integrating with the rest of the system. The IR sensors were tested separately to verify their output values on black and white surfaces.

The L298N motor driver was tested independently to confirm correct direction and speed control via PWM signals from the ESP32.

The OLED display was tested to verify how it display messages and icons in it

6.1.2 Integration Testing

Once individual components were verified, we tested them together. We checked that the ESP32 could read all 5 IR sensors simultaneously without delay, and that it could drive both motors through L298N while also updating the OLED display and maintaining a Firebase connection at the same time.

6.1.3 System Testing

System testing involved running the robot on the actual line track with the full web app. We tested end-to-end scenarios — selecting source and destination on the web app, watching the command reach the robot, and verifying that the robot successfully navigated to the correct destination

6.1.4 User Acceptance Testing (UAT)

We had a few classmates and our guide test the web interface without any instructions to check if it was intuitive enough. The feedback confirmed that the interface is straightforward and easy to use without guidance.

6.2 Test Cases

The following test cases were executed to validate the system. Each test case includes the input, expected output, actual output, and result.

Table 6.1: Hardware test cases

TC	Component	Description	Expected Output	Result
TC-01	ESP32	Verify ESP32 connects to WiFi successfully	Serial Monitor shows 'WiFi connection and IP address, OLED shows Online status	PASS
TC-02	ESP32 + Firebase	Verify ESP32 reads navigation data from Firebase Database	ESP32 reads and prints correct from, to, and directions values	PASS
TC-03	ESP32 + Firebase	Verify ESP32 writes connection status back to Firebase	Firebase shows connectionStatus = 'Online'	PASS
TC-04	ESP32 + L298N	Verify both motors spin in correct direction and speed	Both motors respond correctly to all four direction commands	PASS
TC-05	IR Sensor	Verify all 5 sensors correctly detect black line/white surface	White surface = all sensors 1, Black surface = all sensors 0	PASS
TC-06	PID Logic	Verify robot follows a straight black line accurately	Robot follows straight line without going off track	PASS

TC-07	PID Logic	Verify robot follows curved sections of track	Robot follows curve without going off track (using PID)	PASS
TC-08	IR Sensor	Verify robot detects node (all 5 sensors black)	Serial Monitor prints 'Node Detected' and robot stops	PASS
TC-09	Motors + IR	Verify robot executes correct 90 degree turn at node	Robot stops at node, turns, finds line, and continues	PASS
TC-10	Firestore	Verify visitor details are saved to Firestore on navigation	New document created in visitors collection with correct fields	PASS
TC-11	ESP32 + IR	Verify robot stops safely when line is lost	Robot stops immediately when all sensors read white	PASS
TC-12	OLED Display	Verify OLED displays correct information at each stage	OLED shows boot status, WiFi, IP, and direction info	PASS
TC-13	Send command without Wi-Fi on robot	Robot stays idle, no crash	Robot waited and reconnected on Wi-Fi restore	PASS
TC-14	Full System	Verify complete navigation flow from web to destination	Robot receives command, follows path, and arrives correctly	PASS

Table 6.2: System Test Cases

TC	Component	Description	Expected Output	Result
TC-01	Select N1 as source, N6 as destination	BFS runs and path $N1 \rightarrow N4 \rightarrow N6$ is shown	Correct path calculated and displayed	PASS
TC-02	Click Navigate	Command sent to Firebase, path highlighted on map	Map highlighted, toast message : command sent successfully	PASS
TC-03	Select same source and destination	App should show no path or alert user	Shows empty path handled gracefully	PASS
TC-04	Visitor Log	Search bar correctly filters the visitor table by input character.	Entering “M” successfully isolated relevant rows in the visitor log	PASS
TC-05	Export Navigation Data	Clicking “Export CSV” generates a downloadable file containing recent visitor records	A .csv file was successfully generated and downloaded containing all visitor records.	PASS
TC-06	Admin Dashboard	Verify analytics dashboard displays visitor data	All stats, charts, and tables reflect correct data	PASS

FUTURE ENHANCEMENT

7.1 Future Enhancements

The current version of the Path Guiding Robot works well for its intended purpose autonomously navigating a predefined campus track based on user input from a web app. However, there are several improvements and additions we plan to explore in the future to make the system smarter, more robust, and practically deployable in a real campus environment.

7.1.1 Obstacles Detections and Avoidance

Currently, the robot does not detect obstacles in its path. Adding an ultrasonic sensor (HC-SR04) or an IR proximity sensor at the front of the robot would allow it to pause and wait if a person or object is in the way, and resume once the path is clear. This is the most important safety feature for real-world deployment.

7.1.2 Live Location Tracking on the Web App

Right now, the web app shows the robot's status (Idle / Moving / Reached) but not its exact position on the map at every moment. In future, the robot can update its current node to Firebase at each intersection, and the web app can animate the robot's movement along the path on the campus map in real time, making it much more informative for the user.

7.1.4 Mobile Application

A dedicated Android or iOS mobile app would make it even easier for campus visitors to use the navigation system. The app could use the device's GPS to automatically detect the user's nearest campus node and set it as the starting point, reducing the steps needed to start navigation.

7.1.5 Automatic Charging

To reduce the need for manual battery charging, a small solar panel or a wireless charging pad at the robot's home base could be integrated to keep the robot powered throughout the day.

BIBLIOGRAPHY/ REFERENCES

8.1 References

1) PathGuidingRobot GitHub Repository

<https://github.com/Maulik-Pithiya/PathGuidingRobot.git>

2) User site link

<https://path-guiding-robot.vercel.app/>

3) Admin site link

<https://path-guiding-robot.vercel.app/adminRobot.html>

4) Google Firebase Realtime Database

<https://pathguidedrobot-default-rtdb.firebaseio.com/>

5) ESP32: Getting Started with Firebase (RealtimeDatabase)

<https://randomnerdtutorials.com/esp32-firebase-realtime-database/>

6) Vercel Inc., Vercel Deployment Documentation, 2024

<https://vercel.com/docs>

7) Tailwind Labs, Tailwind CSS Documentation, 2024

<https://tailwindcss.com/docs>

8) Wokwi, Wokwi ESP32 and Arduino Simulator, 2024

<https://wokwi.com>

REPORTING REPORT

Smt. Chandaben Mohanbhai Patel Institute of Computer Applications

Project Guide Reporting Sheet

BCA/ B.Sc.IT

Student(s) Id and Names	23BSIT119 Maulik Pithiya 23BSIT055 Swal Pandya 23BSIT060 Abhishek Parmar
Project Title	Path guiding robot

Sr. No.	Date	Discussion Topic	Suggestions	Signature of Student(s)
1	08-01-26	Initial Project idea discussion	Finalize Project scope	Mt. Pithiya
2	19-01-26	Fixe bird V hard ware components	Refer hardware manual and documents	Mt. Pithiya
3	28-01-26	IR Sensor working principle	Study component of Sensors	Mt. Pithiya
4	10/02/26	Hardware components selection	check compatibility	Mt. Pithiya
5	18/02/26	Software component Part	Add real-time status improve UI	Mt. Pithiya
6	08/3/26	DataBase integration	Test data read/write	Mt. Pithiya
7	10/03/26	Logical Part discussion	implement BFS on website as well	Mt. Pithiya
8	17/03/26	Project Progress review	test all hardware components	Mt. Pithiya
9	24/03/26	ESP32 Fire base communication	Use Library : firebase ESP32	Mt. Pithiya
10	31/03/26	Power manage-ment in hardware	Analyze ^{req.} Voltage and use battery as requirement	Mt. Pithiya
11	16/03/26	Admin Dash-board	Add Report generate Button	Mt. Pithiya

Project Guide Name: Dr. Shailesh Khant

Sign: _____

(Signature)